Étude d'une attaque par fautes sur l'algorithme DES

Rapport de Contrôle Continu - M1 SECRETS

Alexandre François

Mai 2026

Table des matières

1 Introduction **1**

2 Principe de l'attaque par fautes contre le DES **1**

3 Application concrète et extraction de la sous-clé K_{16} **2**

4 Reconstruction de la clé complète **3**

5 Fautes sur les tours précédents (Tours 14 et 13) **3**
5.1 Faute sur le 14ème tour (R_{14}) 3
5.2 Fautes sur les tours médians (12ème et 13ème tours) 4
5.3 Fautes sur les tours 11 à 9 et limites de l'attaque 5

6 Contre-mesures **5**

Résumé

— **Sous-clé K_{16} (48 bits) :** `F5 09 2F E6 EE 7E`
— **Clé complète (64 bits) :** `6E 38 7C DF B0 57 AB 3B`

1 Introduction

L'évaluation de la sécurité physique des circuits cryptographiques est essentielle en cryptanalyse. L'exécution matérielle de l'algorithme Data Encryption Standard (DES) l'expose aux injections de fautes [5]. L'Analyse Différentielle par Fautes (DFA), adaptée au DES par Biham et Shamir en 1997, exploite cette vulnérabilité [1].

L'attaque consiste à induire une perturbation transitoire (par impulsion laser, variation de tension ou rayonnement électromagnétique) durant un cycle d'horloge précis pour provoquer un basculement de bit dans un registre interne [5]. La confrontation du chiffré sain et des chiffrés erronés issus d'un même texte clair permet de modéliser la propagation de l'erreur. Ce rapport analyse cette mécanique algébrique sur le 15ème tour du DES, documente l'extraction de la clé secrète (6E 38 7C DF B0 57 AB 3B), évalue la viabilité des attaques sur les tours intermédiaires et présente les contre-mesures existantes.

2 Principe de l'attaque par fautes contre le DES

La cryptanalyse DFA contre le DES exploite l'architecture symétrique de son réseau de Feistel. L'algorithme scinde le bloc de données de 64 bits en deux moitiés de 32 bits, L et R, traitées sur 16 itérations. Le modèle de faute classique suppose que l'attaquant parvient à corrompre la valeur du registre droit à la fin du 15ème tour, noté R_{15}, introduisant une erreur additive ϵ telle que le registre fauté devient $R^*_{15} = R_{15} \oplus \epsilon$. Expérimentalement, cette perturbation physique transitoire est provoquée par des vecteurs externes tels qu'une impulsion laser, une variation abrupte de la tension

d'alimentation ou une injection électromagnétique. Il est fondamental de noter que la moitié gauche L_{15} demeure intacte lors de cette opération.

Lors du 16ème et dernier tour, les équations standards du réseau de Feistel définissant la transition d'état s'expriment par $L_{16} = R_{15}$ et $R_{16} = L_{15} \oplus f(R_{15}, K_{16})$, où f est la fonction non linéaire du DES et K_{16} la sous-clé de 48 bits du dernier tour. Sous l'effet de la faute injectée, le calcul de l'exécution corrompue dévie mathématiquement pour produire $L^*_{16} = R^*_{15} = R_{15} \oplus \epsilon$ et $R^*_{16} = L_{15} \oplus f(R^*_{15}, K_{16})$. En inversant la permutation finale IP^{-1} sur les deux chiffrés obtenus (le juste C et le faux C^*), le cryptanalyste récupère l'accès direct aux registres (L_{16}, R_{16}) et (L^*_{16}, R^*_{16}).

L'analyse différentielle de ces paires de registres met en évidence une simplification mathématique directe. Le calcul de la différence sur la moitié gauche s'isole ainsi :

$$\Delta L_{16} = L_{16} \oplus L^*_{16} = R_{15} \oplus (R_{15} \oplus \epsilon) = \epsilon$$

Par conséquent, la valeur exacte du masque d'erreur ϵ est directement accessible. Cela permet à l'attaquant de connaître précisément l'erreur injectée sans avoir à la deviner ou à modéliser physiquement la faute.

Parallèlement, la différence sur la moitié droite s'établit comme $\Delta R_{16} = R_{16} \oplus R^*_{16}$. En substituant les termes issus du réseau de Feistel, le terme L_{15} s'annule, isolant la sous-clé K_{16} dans la relation $\Delta R_{16} = f(R_{15}, K_{16}) \oplus f(R^*_{15}, K_{16})$. La fonction f englobe une expansion E, une substitution non linéaire via 8 boîtes S (S-boxes) et une permutation P. La permutation P étant une transformation linéaire, elle peut être inversée pour relier la différence observée directement aux sorties des boîtes S, formant l'équation fondamentale de l'attaque :

$$P^{-1}(\Delta R_{16}) = S(E(R_{15}) \oplus K_{16}) \oplus S(E(R^*_{15}) \oplus K_{16})$$

Pour chaque boîte S_j, cette relation s'écrit sous la forme condensée :

$$S_j(x_j \oplus k_j) \oplus S_j(x^*_j \oplus k_j) = \delta_j$$

où les différences $\Delta x_j = E_j(\Delta L_{16})$ et $\delta_j = P_j^{-1}(\Delta R_{16})$ sont connues de l'attaquant.

La réduction du nombre de clés candidates s'explique par le principe des tiroirs. Une boîte S prend 6 bits en entrée (64 valeurs de k_j) et renvoie 4 bits en sortie (16 valeurs de δ_j). En moyenne, chaque différence de sortie correspond à $64 \div 16 = 4$ clés. La symétrie de l'opération XOR implique que si k_j est une solution, $k_j \oplus \Delta x_j$ l'est également. Le nombre de solutions est donc pair. L'observation expérimentale de δ_j permet d'isoler l'ensemble correspondant, réduisant l'espace de recherche initial de 64 clés à 4 candidats en moyenne (soit une réduction d'un facteur 16).

La propagation de la faute dépend de la fonction d'expansion E, qui duplique 16 des 32 bits d'entrée. L'injection d'une faute d'un bit sur R_{15} entraîne deux scénarios équiprobables :

- **Propagation simple :** Le bit corrompu n'est pas dupliqué par E. L'erreur impacte une seule boîte S, permettant de déterminer 6 bits de la sous-clé K_{16}.
- **Propagation double :** Le bit corrompu est dupliqué par E. L'erreur impacte deux boîtes S adjacentes. La résolution des deux équations permet de déterminer 12 bits de K_{16}.

Une injection isolée ne révélant qu'une fraction de la sous-clé, la reconstitution des 48 bits de K_{16} requiert l'analyse d'un échantillon de x chiffrés fautés. Le ciblage de différentes positions d'injection permet de couvrir l'ensemble des 8 boîtes S et d'assembler la sous-clé complète. La complexité algorithmique dépend linéairement du nombre x de chiffrés fautés analysés. Pour chaque faute, la résolution des équations locales exige au maximum 8 évaluations indépendantes de 2^6 valeurs, soit 2^9 opérations. La complexité totale pour extraire la sous-clé K_{16} s'établit donc à $x \times 2^9$ opérations.

3 Application concrète et extraction de la sous-clé K_{16}

L'application de ce cadre théorique repose sur une méthode d'élimination algorithmique systématique. Le jeu de données analysé comprend un texte clair identifié (`FD 1C 38 2A 5D C8 1C 73`), son chiffré valide (`1A 6F CE 10 24 27 97 D6`) et un échantillon de 32 chiffrés fautés générés sous la même clé.

La procédure débute par l'initialisation de huit ensembles de clés candidates, correspondant aux huit boîtes S, contenant chacun l'intégralité des 64 valeurs possibles pour le fragment k_j. L'algorithme itère ensuite sur les chiffrés fautés. Pour chaque paire (juste/fauté), les différences d'entrée Δx_j et de sortie δ_j sont calculées. L'équation différentielle établie à la section précédente est alors évaluée : tout candidat k_j ne satisfaisant pas cette relation est définitivement supprimé de son ensemble.

L'intersection successive des candidats valides à travers l'analyse des 32 fautes permet d'éliminer les faux positifs inhérents aux collisions des boîtes S [1]. Cette convergence isole de manière déterministe une valeur unique pour chaque bloc. L'assemblage final des 8 fragments révèle la valeur exacte de la sous-clé K_{16} : `F5 09 2F E6 EE 7E`.

La complexité totale pour extraire la sous-clé K_{16} avec 32 chiffrés fautés s'établit donc à $32 \times 2^9 = 2^{14}$ opérations.

4 Reconstruction de la clé complète

La sous-clé K_{16} ne contient que 48 bits sur les 56 bits de la clé interne (elle-même issue de la clé complète de 64 bits). Pour retrouver les 8 bits manquants, il faut donc inverser le mécanisme de génération des clés. Lors de cette étape (Key Schedule), la table de permutation PC2 ne sélectionne que 48 bits dans les registres C_{16} et D_{16}, laissant délibérément de côté 8 positions précises définies par le standard DES.

La stratégie consiste à tester toutes les possibilités (force brute) uniquement sur ces 8 bits manquants. L'espace de recherche étant limité à $2^8 = 256$ combinaisons, l'exécution de cette étape est immédiate. Pour chaque essai, les 8 bits sont insérés aux positions ignorées par la table PC2, ce qui permet de reconstituer l'état interne de 56 bits. Une propriété mathématique du DES simplifie cette étape : au fil des 16 tours, les décalages circulaires successifs des registres totalisent exactement 28 crans. Les registres faisant 28 bits, ils effectuent un tour complet sur eux-mêmes. Ainsi, l'état de 56 bits que l'on vient de reconstituer correspond directement à l'état initial de la clé avant le chiffrement. Le script inverse ensuite la table PC1 pour replacer ces 56 bits dans un format de 64 bits. L'algorithme calcule alors la parité : pour chaque bloc de 7 bits, il ajoute un 8ème bit (0 ou 1) pour s'assurer que le nombre total de bits à "1" dans l'octet soit impair, comme l'exige le standard DES. Enfin, la clé de 64 bits ainsi formée est testée : le script l'utilise pour déchiffrer le cryptogramme connu. Sur les 256 clés testées, une seule permet de retrouver le texte clair attendu (FD 1C 38 2A 5D C8 1C 73). Cette dernière vérification prouve que la clé trouvée est la bonne : 6E 38 7C DF B0 57 AB 3B.

À titre d'exemple, le premier octet de la clé (0x6E) s'écrit 01101110 en binaire. Il contient exactement cinq bits à "1", ce qui respecte bien la règle de parité impaire exigée par le NIST [3].

En conclusion, la complexité totale pour finaliser l'extraction de la clé complète s'établit à $x \cdot 2^9 + 2^8$ opérations.

5 Fautes sur les tours précédents (Tours 14 et 13)

En supposant que l'attaquant provoque une faute plus tôt dans l'algorithme, notamment sur les registres R_{14} ou R_{13}, les équations de l'attaque diffèrent. Cette approche accroît la difficulté mathématique en raison de l'effet d'avalanche du DES. Cependant, des recherches récentes modélisant la propagation des erreurs démontrent qu'il reste possible de recouvrer la clé avec un nombre très restreint de chiffrés [2].

5.1 Faute sur le 14ème tour (R_{14})

Une faute injectée à la fin du 14ème tour modifie le registre droit tel que $R_{14}^* = R_{14} \oplus \epsilon$ (où ϵ correspond à un unique bit inversé). La perturbation se propage à travers deux tours consécutifs de la fonction non linéaire f :

— **Impact au tour 15 :** La moitié gauche, copie du registre droit précédent, contient l'erreur : $\Delta L_{15} = L_{15} \oplus L_{15}^* = \epsilon$. Le calcul du registre droit est déterminé par la fonction f :

$$R_{15}^* = L_{14} \oplus f(R_{14} \oplus \epsilon, K_{15})$$

La différence générée à l'issue de ce tour est donc :

$$\Delta R_{15} = f(R_{14}, K_{15}) \oplus f(R_{14} \oplus \epsilon, K_{15})$$

— **Impact au tour 16 (chiffré final) :** La moitié gauche finale correspond à la différence du tour 15, ce qui permet d'obtenir :

$$\Delta L_{16} = \Delta R_{15}$$

La moitié droite dépend à la fois de la faute initiale ϵ (transmise via L_{15}^*) et de la nouvelle différence induite par la fonction f. La différence sur la moitié droite finale se dérive explicitement à partir des expressions du tour 16 non fauté et fauté :

$$\Delta R_{16} = R_{16} \oplus R_{16}^* = \big(L_{15} \oplus f(R_{15}, K_{16})\big) \oplus \big(L_{15}^* \oplus f(R_{15}^*, K_{16})\big)$$

En regroupant les termes et en exploitant $L_{15} \oplus L_{15}^* = \epsilon$, on obtient la relation finale :

$$\Delta R_{16} = \epsilon \oplus f(R_{15}, K_{16}) \oplus f(R_{15}^*, K_{16})$$

Contrairement à l'attaque sur le 15ème tour, la valeur de ϵ n'est pas directement observable depuis le chiffré final : elle affecte R_{14}, un registre interne inaccessible à l'attaquant. Le système d'équations fait donc intervenir ϵ comme une inconnue supplémentaire, aux côtés des sous-clés K_{15} et K_{16}.

Les étapes pour retrouver la clé sont les suivantes :

1. **Hypothèse sur l'erreur initiale :** En postulant que l'erreur ϵ induite correspond à l'inversion d'un unique bit, l'espace de recherche de la faute se limite à 32 positions possibles dans R_{14}, chacune induisant une propagation différente via E, S, P.
2. **Ciblage de K_{16} :** L'équation du 16ème tour est réagencée pour exprimer l'erreur :

$$\epsilon = \Delta R_{16} \oplus f(R_{15}, K_{16}) \oplus f(R_{15}^*, K_{16})$$

Par construction de la table d'expansion E, un unique bit inversé dans R_{14} active 1 ou 2 boîtes S au tour 15, produisant 4 ou 8 bits actifs dans ΔR_{15} après la permutation P. Ces bits, réexpansés par E au tour 16, activent entre 1 et 4 boîtes S, impliquant 6 à 24 bits de contraintes sur K_{16} [3].

3. **Filtrage par boîte S et intersection :** L'attaquant procède boîte S par boîte S en testant les 2^6 combinaisons de clé locale. Seules les valeurs générant une erreur ϵ avec exactement un bit à 1 sont conservées. Les candidats valides sont combinés pour former une clé potentielle sur 48 bits. En répétant cette opération avec x chiffrés fautés, l'intersection des résultats permet d'éliminer les faux positifs jusqu'à isoler la valeur unique et exacte de K_{16} et de ϵ.
4. **Reconstruction de la clé complète :** Une fois K_{16} isolée, la clé complète de 64 bits est retrouvée selon la procédure décrite en section 4, par recherche exhaustive sur les 8 bits non sélectionnés par PC-2 (2^8 évaluations).

Analyse de complexité Le calcul pour le 16ème tour nécessite $8 \times 2^6 = 2^9$ opérations par chiffré. Pour un échantillon de x chiffrés, la complexité totale est donc $x \cdot 2^9 + 2^8$. Bien que ϵ soit initialement inconnu, l'algorithme procède à une recherche conjointe sur l'espace des sous-clés K_{16}. Pour chaque clé candidate, une valeur de ϵ est déduite. Seules les sous-clés K_{16} générant une erreur ϵ conforme au modèle physique de faute sur l'intégralité de l'échantillon sont maintenues. Ce processus d'intersection garantit la convergence de l'espace des candidats vers la sous-clé exacte.

5.2 Fautes sur les tours médians (12ème et 13ème tours)

L'injection d'une faute sur les registres R_{12} ou R_{13} entraîne une propagation de l'erreur sur respectivement quatre et trois itérations de la fonction f. L'approche proposée par Ma et al. modélise cette diffusion en fonction du nombre de boîtes S impactées (une ou deux) par l'inversion d'un bit unique [2].

Pour une faute au 13ème tour, la méthode repose sur l'établissement d'une table d'états de propagation intermédiaires et de gabarits (*state templates*). La comparaison expérimentale avec ces gabarits permet de localiser l'erreur injectée et de déduire la sous-clé par intersection, nécessitant l'analyse de seulement 2 à 3 paires de chiffrés (justes et fautés) [2].

Pour une faute au 12ème tour, l'attaque exploite un biais probabiliste : les modèles démontrent que certaines différences d'états intermédiaires ont une probabilité prédictible (supérieure ou égale à 1/8) de valoir zéro. Un algorithme de distinction sépare ensuite la clé correcte des clés incorrectes (dont la distribution est uniforme), permettant de recouvrer la clé complète à partir de 4 paires de chiffrés [2].

5.3 Fautes sur les tours 11 à 9 et limites de l'attaque

Pour les injections de fautes situées entre le 11ème et le 9ème tour, la diffusion de l'erreur ne permet plus l'utilisation de gabarits déterministes. L'analyse s'appuie sur les modèles statistiques de Rivain, fondés sur des distingueurs par maximum de vraisemblance ou par distance euclidienne. Ces méthodes exploitent le biais résiduel de la distribution des différences pour isoler la sous-clé [4]. La complexité en données augmente avec la profondeur du tour ciblé :

- **11ème tour :** L'attaque requiert un minimum de 11 chiffrés fautés dans le modèle d'erreur sur un bit à position choisie.
- **10ème tour :** Le volume minimal de données s'élève à 290 chiffrés fautés.
- **9ème tour :** L'extraction de la clé nécessite $3,4 \cdot 10^5$ chiffrés fautés pour distinguer la clé correcte du bruit uniforme.

La faisabilité pratique de cette cryptanalyse est limitée au 9ème tour. Pour toute injection effectuée en amont (8ème tour ou antérieur), l'erreur traverse au moins 8 itérations de la fonction f. À ce stade, la diffusion de l'algorithme produit une distribution des différences statistiquement uniforme et indiscernable d'un comportement aléatoire. Le volume de chiffrés requis excède alors 10^8 paires, rendant l'attaque inefficace par rapport à une recherche exhaustive standard.

6 Contre-mesures

La protection matérielle contre l'Analyse Différentielle par Fautes (DFA) repose sur l'introduction de redondances pour vérifier l'intégrité du calcul. L'article de Rivain [4] identifie trois méthodes principales :

- **Doublement du calcul :** L'algorithme de chiffrement est exécuté intégralement deux fois avec le même texte clair et la même clé. Le circuit compare les deux résultats obtenus (C_1 et C_2) et bloque la sortie du cryptogramme si une divergence est détectée ($C_1 \neq C_2$).
  Complexité : Cette approche engendre un surcoût temporel de 100 %. L'exécution complète est requise pour garantir qu'aucune faute injectée dans les tours initiaux ne se propage vers les tours terminaux.
- **Vérification par déchiffrement :** L'opération de déchiffrement est appliquée au cryptogramme obtenu. Si le résultat diffère du texte clair initial, le système détecte la faute et bloque la sortie.
  Complexité : Surcoût temporel de 100 %.
- **Redondance au niveau des opérations :** Des vérifications de cohérence sont intégrées directement au niveau des opérations internes de l'algorithme.
  Complexité : Le surcoût temporel est quasi nul, mais le surcoût matériel reste comparable à celui d'un doublement du calcul.

Références

[1] Eli Biham and Adi Shamir. Differential fault analysis of secret key cryptosystems. https://cs-people.bu.edu/tromer/courses/infosec11/Biham%20Shamir%201997%20---%20Differential%20Fault%20Analysis%20of%20Secret%20Key%20Cryptosystems.pdf, 1997.

[2] Xiangliang Ma, Lizhen Zhang, Liji Wu, Xia Li, Xiangmin Zhang, Bing Li, and Yuling Liu. Differential fault analysis on 3DES middle rounds based on error propagation. https://ieeexplore.ieee.org/document/9754321, 2022.

[3] National Institute of Standards and Technology (NIST). Fips pub 46-3 : Data encryption standard (des). https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf, 1999.

[4] Matthieu Rivain. Differential fault analysis on DES middle rounds. https://matthieurivain.com/files/ches09a.pdf, 2009.

[5] Wikipedia contributors. Differential fault analysis — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Differential_fault_analysis, 2026. [Accédé le 9-Mai-2026].